Operating System

Introduction

Comprehensive Study Notes

Process | Memory | I/O | File Management | Interrupts | DMA

Designed for: University Exams | Competitive Exams | Revision

Includes: Definitions • Examples • Gantt Charts • Comparison Tables • Exam Notes

Study Smart | Score Better

Table of Contents

1. Process Management

- FCFS (First Come First Serve) Scheduling
- Round Robin Scheduling
- Priority-Based Scheduling
- Comparison of Scheduling Algorithms

2. Memory Management

- Segmentation
- Paging
- Virtual Memory
- Comparison: Segmentation vs Paging vs Virtual Memory

3. I/O Management

- Concept of I/O Port
- Types of I/O Ports
- Data Transfer Process

4. File Management

- FAT (File Allocation Table)
- File Handling Functions
- Common File Operations

5. Interrupts

- Software Interrupts
- Hardware Interrupts
- Interrupt Handling Mechanism

6. Addressing Concepts

- I/O Based Addresses
- Memory Based Addresses
- I/O Mapped vs Memory Mapped Addressing

7. DMA (Direct Memory Access)

- DMA Controller
- DMA Transfer Process
- DMA Channels
- Types of DMA Transfer

8. Glossary of Terms

9. Revision Summary & Conclusion

1 Process Management

FCFS | Round Robin | Priority-Based Scheduling

Introduction to Process Management

Process management is a core function of an Operating System (OS) that controls the execution of processes. A **process** is a program in execution. The OS decides which process runs at any given time through **CPU Scheduling algorithms**. The goal is to maximize CPU utilization, throughput, and minimize waiting time and turnaround time.

Key Terms:

- **Burst Time (BT):** CPU time required by a process.
- **Arrival Time (AT):** Time when process arrives in ready queue.
- **Completion Time (CT):** Time when process finishes execution.
- **Turnaround Time (TAT):** CT - AT
- **Waiting Time (WT):** TAT - BT
- **Response Time:** Time from arrival until first CPU allocation.

1.1 FCFS – First Come First Serve Scheduling

Definition: FCFS Scheduling

FCFS is the simplest CPU scheduling algorithm where the process that arrives first in the ready queue gets the CPU first. It is a non-preemptive algorithm — once a process starts execution, it runs to completion.

Working Mechanism

Processes are placed in a queue in the order they arrive. The CPU serves them one by one from the front of the queue. If two processes arrive at the same time, they are arranged by process ID or order of submission.

Characteristics

- Non-preemptive: A running process is not interrupted.
- Simple queue (FIFO) data structure.

FIFO = First In, First Out

The process that enters the queue first gets executed first.

Just like a queue at a ticket counter — whoever arrives first is served first.

- Easy to implement and understand.
- Poor average waiting time in most cases (Convoy Effect).

Convoy Effect: A situation in FCFS scheduling where a long process occupies the CPU first, forcing shorter processes to wait behind it, resulting in increased waiting time and reduced system efficiency.

Example: If P1 has burst time = 20ms and P2, P3 have burst time = 2ms each,

P2 and P3 must wait 20ms even though they only need 2ms — like a slow truck

blocking fast cars on a single-lane road.

- No starvation — every process gets CPU eventually.

Solved Example

Question: Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT). Schedule them using the FCFS (First Come First Serve) scheduling algorithm. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process. Also draw the Gantt Chart and find the Average TAT and Average WT.

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	4
P2	1	3
P3	2	5
P4	3	2

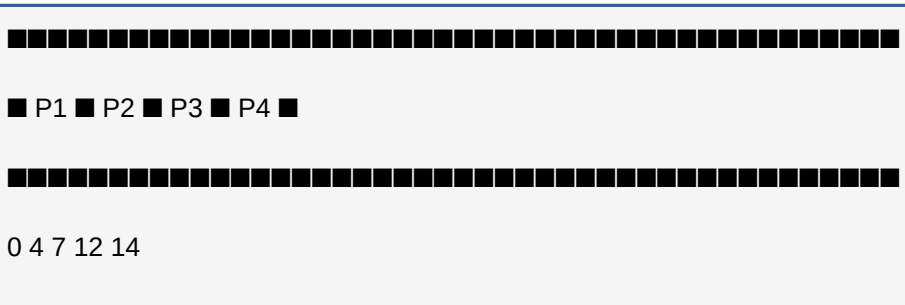
Given: Algorithm = FCFS | Mode = Non-Preemptive

Find: CT, TAT = CT – AT, WT = TAT – BT, Avg TAT, Avg WT

Solution:

Process	Arrival Time (AT)	Burst Time (BT)	Start Time	Completion Time (CT)	TAT = CT–AT	WT = TAT–BT
P1	0	4	0	4	4	0
P2	1	3	4	7	6	3
P3	2	5	7	12	10	5
P4	3	2	12	14	11	9

Gantt Chart:



Average TAT = (4 + 6 + 10 + 11) / 4 = 31 / 4 = 7.75 units

Average WT = (0 + 3 + 5 + 9) / 4 = 17 / 4 = 4.25 units

Advantages & Disadvantages

Advantages

✓ Simple to implement using a FIFO queue.

✓ No complex decision-making required.

✓ Fair in the sense every process eventually executes.

✓ No overhead of context switching mid-process.

Disadvantages

✗ Convoy Effect: Short processes wait behind long ones.

✗ High average waiting time.

✗ Not suitable for time-sharing systems.

✗ Poor response time for interactive processes.

★ EXAM FOCUS

- FCFS is NON-PREEMPTIVE.
- Convoy Effect is the main drawback of FCFS.
- TAT = CT - AT; WT = TAT - BT
- Average WT is always higher in FCFS compared to SJF.
- If all processes arrive at time 0, FCFS and SJF give same result only if burst times are equal.

1.2 Round Robin (RR) Scheduling

Definition: Round Robin Scheduling

Round Robin is a preemptive CPU scheduling algorithm where each process is assigned a fixed time slice called a **Time Quantum (TQ)**. After each quantum, the process is moved to the back of the ready queue, and the next process gets the CPU.

Working Mechanism

The OS maintains a circular ready queue. Each process gets CPU for exactly one time quantum. If the process finishes within the quantum, it exits; otherwise it is preempted and re-queued. This continues until all processes finish.

Characteristics

- Preemptive scheduling algorithm.
- Time Quantum (TQ) is the key parameter — affects performance.
- Best suited for time-sharing and interactive systems.
- No starvation — every process gets equal CPU time in rotation.
- Context switching overhead increases with smaller TQ.
- If TQ is very large → behaves like FCFS; very small → overhead increases.

Solved Example

Question: Consider the following set of 4 processes with their Arrival Time (AT) and Burst Time (BT). Schedule them using the Round Robin (RR) scheduling algorithm with Time Quantum (TQ) = 2. Calculate the Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and Response Time (RT) for each process. Also draw the Gantt Chart and find the Average TAT and Average WT.

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	5
P2	1	4
P3	2	2
P4	4	2

Given: Algorithm = Round Robin | Time Quantum (TQ) = 2 | Mode = Preemptive

Find: CT, TAT = CT – AT, WT = TAT – BT, RT = First CPU time – AT

Average TAT, Average WT

Solution:

Formulae Used:

TAT = CT – AT

WT = TAT – BT

RT = First CPU allocated time – AT

Step-by-Step Execution Trace (Ready Queue → Running):

Step	Process	Start Time	End Time	CPU Used (this slot)	Remaining Burst Time	Status
1	P1	0	2	2	5 – 2 = 3	Preempted
2	P2	2	4	2	4 – 2 = 2	Preempted
3	P3	4	6	2	2 – 2 = 0	Completed ✓
4	P1	6	8	2	3 – 2 = 1	Preempted
5	P4	8	10	2	2 – 2 = 0	Completed ✓
6	P2	10	12	2	2 – 2 = 0	Completed ✓
7	P1	12	13	1	1 – 1 = 0	Completed ✓

Gantt Chart:

■■■

0 2 4 6 8 10 12 13

Ready Queue order: P1 → P2 → P3 → P1 → P4 → P2 → P1

Final Results Table:

Process No.	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	TAT = CT–AT	WT = TAT–BT	RT = 1st CPU–AT
P1	0	5	13	13	8	0 – 0 = 0
P2	1	4	12	11	7	2 – 1 = 1
P3	2	2	6	4	2	4 – 2 = 2
P4	4	2	10	6	4	8 – 4 = 4
Average	—	—	—	**8.50**	**5.25**	—

Average TAT = (13 + 11 + 4 + 6) / 4 = 34 / 4 = 8.50 units

Average WT = (8 + 7 + 2 + 4) / 4 = 21 / 4 = 5.25 units

Advantages & Disadvantages

Advantages

✓ Fair allocation — all processes get equal CPU time.

✓ No starvation.

✓ Good response time for short processes.

✓ Best for time-sharing systems.

✓ Simple to implement.

Disadvantages

✗ High context-switching overhead due to frequent CPU switching.

✗ Performance depends heavily on the selection of Time Quantum; too small increases overhead, too large makes RR behave like FCFS.

✗ Not optimal for batch processing.

✔ KEY POINTS

- Choosing the right Time Quantum is critical.
- Rule of thumb: TQ should be slightly larger than the average burst time.
- Response Time in RR ≈ (n-1) × TQ, where n = number of processes.

★ EXAM FOCUS

- Round Robin is PREEMPTIVE — meaning the CPU can be taken away from a running process before it finishes.
- Time Quantum too small → too many context switches (overhead).
- Time Quantum too large → degenerates to FCFS.
- RR provides best RESPONSE TIME among basic scheduling algorithms.
- No starvation in Round Robin — every process gets CPU in rotation.

1.3 Priority-Based Scheduling

Definition: Priority Scheduling

Priority scheduling assigns a priority number to each process. The CPU is allocated to the process with the highest priority. It can be either preemptive (higher priority process can preempt running process) or non-preemptive.

Working Mechanism

Each process is assigned a priority (commonly lower number = higher priority). The scheduler selects the process with the best priority from the ready queue. In the **preemptive** version, if a new higher-priority process arrives, the current process is paused. In the **non-preemptive** version, the current process runs to completion.

Priority Assignment Methods

Method	Description
Static Priority	Fixed at process creation; does not change during execution.
Dynamic Priority	Changes over time; aging technique used to prevent starvation.
Internal Priority	Determined by OS based on I/O time, memory requirements.
External Priority	Set by users/administrators.

Example 1 — Non-Preemptive Priority Scheduling

Question: Consider the following processes. Schedule using Priority Scheduling (Non-Preemptive). Lower number = Higher priority. Calculate CT, TAT, WT and draw Gantt Chart.

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	4	3
P2	1	3	1 ← Highest Priority
P3	2	5	4 ← Lowest Priority
P4	3	2	2

Given: Algorithm = Priority Scheduling | Mode = Non-Preemptive

Lower number = Higher priority

Find: CT, TAT = CT – AT, WT = TAT – BT, Avg TAT, Avg WT

Solution — Step-by-Step Logic:

At time 0: Only P1 has arrived → P1 starts (no choice).

P1 runs from 0 → 4 (Non-preemptive: cannot stop even though P2 arrives at t=1)

At time 4: P2 (Priority 1), P3 (Priority 4), P4 (Priority 2) are all waiting.

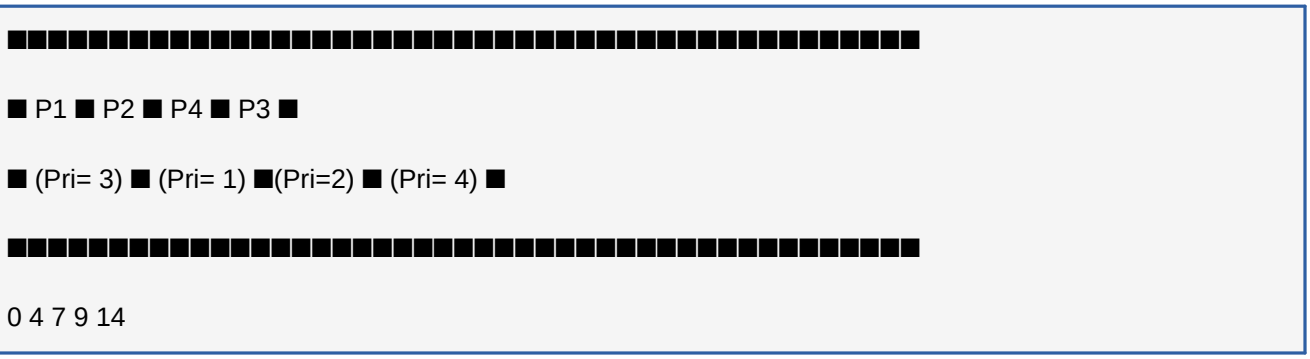
→ Pick highest priority = P2 (Priority 1) → P2 runs 4 → 7

At time 7: P3 (Priority 4) and P4 (Priority 2) are waiting.

→ Pick highest priority = P4 (Priority 2) → P4 runs 7 → 9

At time 9: Only P3 remains → P3 runs 9 → 14

Gantt Chart:



Results Table:

Process	AT	BT	Priority	Start Time	CT	TAT = CT–AT	WT = TAT–BT
P1	0	4	3	0	4	4	0
P2	1	3	1	4	7	6	3
P3	2	5	4	9	14	12	7
P4	3	2	2	7	9	6	4
Avg	—	—	—	—	—	**7.0**	**3.5**

Average TAT = (4 + 6 + 12 + 6) / 4 = 28 / 4 = 7.0 units

Average WT = (0 + 3 + 7 + 4) / 4 = 14 / 4 = 3.5 units

Example 2 — Preemptive Priority Scheduling

Question: Consider the following processes. Schedule using Priority Scheduling (Preemptive). Lower number = Higher priority. If a new process arrives with higher priority than the running process, the CPU is taken away immediately. Calculate CT, TAT, WT and draw Gantt Chart.

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	4	3
P2	1	3	1 ← Highest Priority
P3	2	5	4 ← Lowest Priority
P4	3	2	2

Given: Algorithm = Priority Scheduling | Mode = Preemptive

Lower number = Higher priority

Find: CT, TAT = CT – AT, WT = TAT – BT, Avg TAT, Avg WT

Solution — Step-by-Step Logic:

t = 0: Only P1 arrived → P1 starts running. (P1 remaining = 4)

t = 1: P2 arrives with Priority 1 (higher than P1 Priority 3).

→ P1 is PREEMPTED. P1 remaining = 3. P2 starts running.

t = 2: P3 arrives with Priority 4 (lower than P2 Priority 1).

→ No preemption. P2 continues.

t = 3: P4 arrives with Priority 2 (lower than P2 Priority 1).

→ No preemption. P2 continues.

t = 4: P2 completes (ran 1→4 = 3 units). CT of P2 = 4.

Ready queue: P1(Pri=3, rem=3), P3(Pri=4, rem=5), P4(Pri=2, rem=2)

→ Highest priority = P4 (Priority 2). P4 starts running.

t = 6: P4 completes (ran 4→6 = 2 units). CT of P4 = 6.

Ready queue: P1(Pri=3, rem=3), P3(Pri=4, rem=5)

→ Highest priority = P1 (Priority 3). P1 resumes.

t = 9: P1 completes (ran 6→9 = 3 units). CT of P1 = 9.

Ready queue: P3(Pri=4, rem=5)

→ Only P3 left. P3 runs.

t = 14: P3 completes. CT of P3 = 14.

Gantt Chart:

■■■



0 1 4 6 9 14

P1 is preempted at t=1 when P2 arrives (higher priority).

P1 resumes at t=6 after P4 completes.

Results Table:

Process	AT	BT	Priority	CT	TAT = CT−AT	WT = TAT−BT	Note
P1	0	4	3	9	9	5	Preempted at t=1; resumed at t=6
P2	1	3	1	4	3	0	Ran continuously 1→4
P3	2	5	4	14	12	7	Waited longest (lowest priority)
P4	3	2	2	6	3	1	Ran after P2 completed
Avg	—	—	—	—	**6.75**	**3.25**	—

Average TAT = (9 + 3 + 12 + 3) / 4 = 27 / 4 = 6.75 units

Average WT = (5 + 0 + 7 + 1) / 4 = 13 / 4 = 3.25 units

Comparison: Non-Preemptive vs Preemptive Priority Scheduling

Feature	Preemptive Priority	Non-Preemptive Priority
CPU Allocation	If a higher priority process arrives, CPU is immediately taken and given to it.	Once CPU is given, process holds it till burst time completes, even if higher priority arrives.
Flexibility	More flexible — responds instantly to high priority processes.	Rigid — no interruption once execution begins.
Waiting Time (High Priority)	Waiting time for highest priority process is always zero.	Waiting time for highest priority process may not be zero.
Cost & Switching	Expensive; more context switches waste time.	Cheaper and faster; less switching required.
Use Case	Useful where high priority processes cannot be kept waiting. Ex: Real-time OS, medical systems.	Used where waiting does not cause serious issues. Ex: Batch processing, hardware applications.

Starvation & Aging

Starvation is the main problem with priority scheduling — low-priority processes may wait indefinitely if high-priority processes keep arriving. **Aging** is the solution: the priority of waiting processes is gradually increased over time so they eventually get the CPU.

Advantages & Disadvantages

Advantages

✓ Important tasks are handled first.

✓ Flexible — priority can reflect real-world urgency.

✓ Preemptive version improves response for critical processes.

✓ Used in real-time systems.

Disadvantages

✗ Starvation of low-priority processes.

✗ Indefinite blocking without aging mechanism.

✗ Priority inversion problem in some systems.

✗ More complex implementation than FCFS.

★ EXAM FOCUS

- Priority Scheduling can be PREEMPTIVE or NON-PREEMPTIVE.
- Main problem: STARVATION. Solution: AGING.
- Priority Inversion: Low-priority process holds resource needed by high-priority process.
- In most systems, lower number = higher priority (but check the question!)
- If two processes have same priority, FCFS is used as a tiebreaker.

1.4 Comparison of Scheduling Algorithms

Feature	FCFS	Round Robin	Priority
Type	Non-Preemptive	Preemptive	Both
Criteria	Arrival Order	Time Quantum	Priority Number
Starvation	No	No	Yes (low priority)
Overhead	Low	High (context switches)	Medium
Fairness	Moderate	High	Low (low-priority)
Best For	Batch Systems	Time-Sharing	Real-Time Systems
Convoy Effect	Yes	No	Possible
Response Time	Poor	Best	Good for high priority
Implementation	Simple	Moderate	Complex

2 Memory Management

Segmentation | Paging | Virtual Memory

Introduction to Memory Management

Memory management is the OS function responsible for managing the primary memory (RAM). It tracks each memory location, allocates memory to processes when they need it, and deallocates it when done. The goal is to maximize memory utilization and allow multiprogramming.

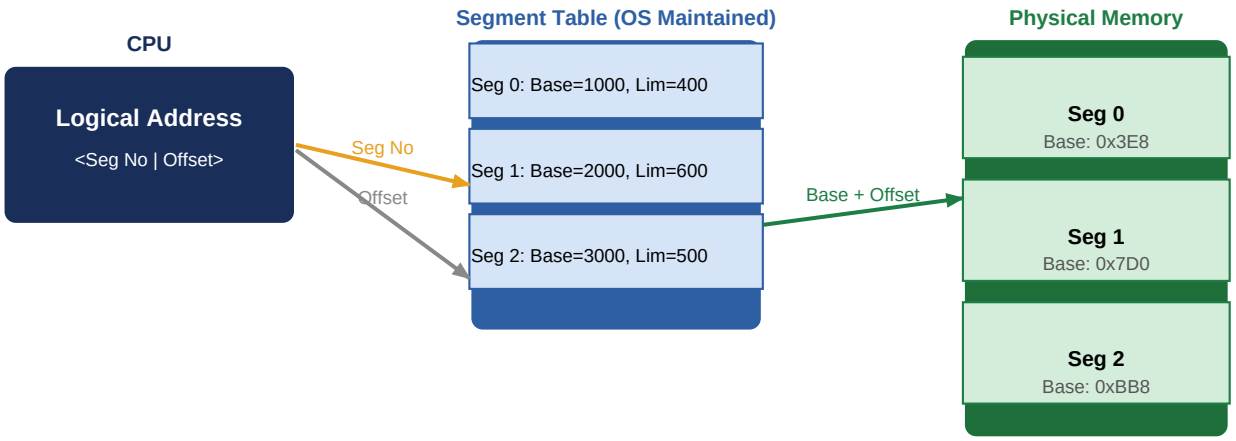
2.1 Segmentation

Definition: Segmentation

Segmentation is a memory management technique where a process is divided into variable-sized logical units called **segments**. Each segment corresponds to a logical unit of a program such as the code, stack, heap, or data section.

Architecture & Working

Each segment has a **segment number** and an **offset**. A **Segment Table** is maintained by the OS, which maps each segment number to its physical base address and limit (length). The logical address consists of two parts: **<Segment Number, Offset>**.



Address Translation Process

Logical Address = <Segment Number (s), Offset (d)>

Step 1: Extract segment number (s) and offset (d) from logical address.

Step 2: Use (s) to index into the Segment Table.

Step 3: Check if offset (d) < Limit of segment → if not, SEGMENTATION FAULT.

Step 4: Physical Address = Base[s] + d

Example: Logical Address = <2, 100>

Segment 2 has Base=3000, Limit=500

Since 100 < 500 → Physical Address = 3000 + 100 = 3100

Segment Table Structure

Segment No.	Base Address	Limit (Size)
0 (Code)	1000	400
1 (Data)	2000	600
2 (Stack)	3000	500
3 (Heap)	4000	300

Advantages & Disadvantages

Advantages

✓ Logical view of memory matches programmer's view.

✓ Segments can grow/shrink independently.

✓ Protection and sharing at segment level.

✓ No internal fragmentation within segments.

✓ Supports modular programming.

Disadvantages

✗ External fragmentation (variable-size segments).

✗ Complex memory allocation algorithms needed.

✗ Segment table maintenance overhead.

✗ Difficult to find a contiguous hole for large segments.

★ EXAM FOCUS

- Segmentation has EXTERNAL FRAGMENTATION, NOT internal.
- Logical address in segmentation: <Segment Number, Offset>
- If Offset >= Limit → Segmentation Fault (protection violation).
- Segmentation supports SHARING and PROTECTION at the segment level.
- Compaction can reduce external fragmentation but is expensive.

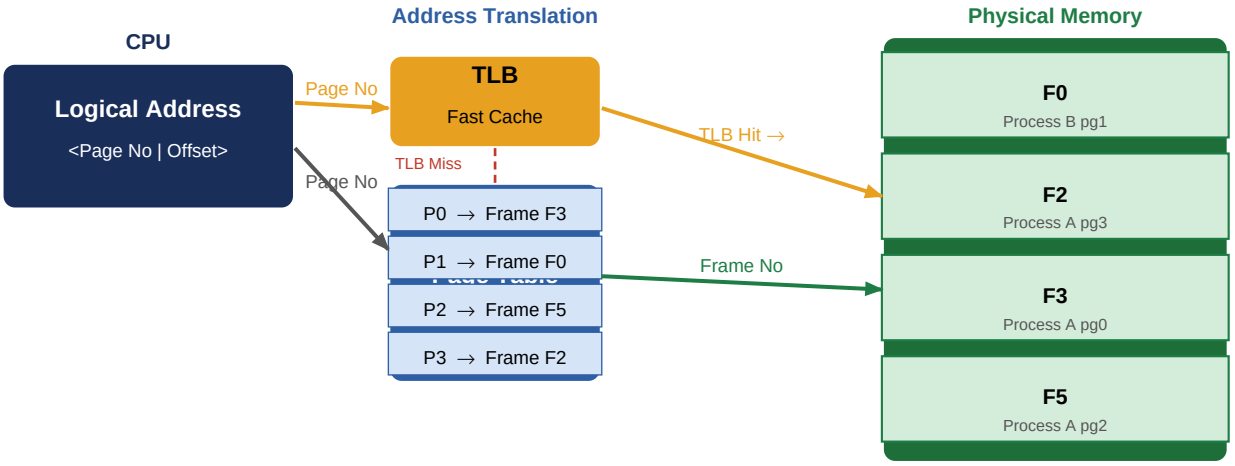
2.2 Paging

Definition: Paging

Paging is a memory management technique where physical memory is divided into fixed-size blocks called **frames** and logical memory is divided into same-size blocks called **pages**. The OS maps pages to frames using a **Page Table**.

Architecture & Working

When a process is loaded, its pages are placed into any available frames (not necessarily contiguous). The OS maintains a Page Table per process that maps page numbers to frame numbers. A logical address is divided into: **<Page Number (p), Page Offset (d)>**.



Address Translation Process

Given: Logical Address Size = 2^n bits, Page Size = 2^k bytes

Page Number bits = n - k | Offset bits = k

Step 1: Divide logical address into page number (p) and offset (d).

Step 2: Use page number (p) to index into Page Table → get Frame Number (f).

Step 3: Physical Address = (f × Page Size) + d

Example: Page size = 1KB (2^10), Logical Address = 2100

Page Number = 2100 / 1024 = 2 (integer), Offset = 2100 mod 1024 = 52

If Page 2 maps to Frame 5 → Physical Address = 5×1024 + 52 = 5172

Translation Lookaside Buffer (TLB)

The TLB is a fast cache memory that stores recent page-to-frame mappings. On each memory access, TLB is checked first (**TLB hit**). If not found (**TLB miss**), the page table in main memory is accessed. TLB significantly reduces memory access time.

Advantages & Disadvantages

Advantages

✓ No external fragmentation.

✓ Simple memory allocation — any free frame can be used.

✓ Logical and physical memory are decoupled.

✓ Easy to implement protection and sharing.

✓ Efficient with TLB for fast address translation.

Disadvantages

✗ Internal fragmentation in the last page of a process.

✗ Page table can be large for large address spaces.

✗ Two memory accesses per logical access (without TLB).

✗ TLB replacement and management overhead.

★ EXAM FOCUS

- Paging eliminates EXTERNAL fragmentation but causes INTERNAL fragmentation.
- Page size = Frame size (always).
- Logical address = <Page Number, Offset> | Physical = Frame × PageSize + Offset
- TLB (Translation Lookaside Buffer) speeds up paging.
- Effective Access Time (EAT) = hit ratio × TLB time + (1-hit ratio) × (TLB + 2×memory time)
- Two-level paging is used for large logical address spaces.

2.3 Virtual Memory

Definition: Virtual Memory

Virtual memory is a memory management technique that allows a process to execute even if it is not completely loaded into physical memory. It creates an illusion of a very large memory by using disk space as an extension of RAM.

Working Mechanism

Virtual memory is implemented using **Demand Paging**. Pages of a process are loaded into memory only when they are needed (*on demand*). Pages not currently needed are stored on disk in a **swap space**. When a needed page is not in memory, a **page fault** occurs.

Page Fault Handling

Step 1: Process accesses a page not in memory → Page Fault occurs.

Step 2: OS saves current process state.

Step 3: OS finds the page on disk (in swap space).

Step 4: If a free frame is available, load the page into it.

Step 5: If no free frame → PAGE REPLACEMENT algorithm is invoked.

Step 6: Update page table to reflect new mapping.

Step 7: Resume the process from the interrupted instruction.

Page Replacement Algorithms

When a page fault occurs and no free frame is available, the OS replaces an existing page using one of the following algorithms:

- **FIFO (First In First Out):** Replaces the oldest page in memory. Drawback: Belady's Anomaly possible.
- **LRU (Least Recently Used):** Replaces the page not used for the longest time. Drawback: Requires tracking usage; complex — OS must keep a record of when each page was last used, making it difficult and expensive to implement.
- **Optimal:** Replaces the page not needed for the longest future time. Drawback: Impractical — needs future knowledge.
- **LFU (Least Frequently Used):** Replaces the page used least number of times. Drawback: Recently loaded pages may be replaced.

Belady's Anomaly: A strange behaviour in FIFO where increasing the number of frames increases the number of page faults instead of decreasing them.

→ More memory = More page faults (counter-intuitive!)

→ Occurs only in FIFO. LRU and Optimal are immune to it.

Thrashing

Thrashing occurs when the OS spends more time swapping pages in and out of memory than executing processes. This happens when there are too many processes competing for limited frames. Solutions include **Working Set Model** and reducing the degree of multiprogramming.

Advantages & Disadvantages

Advantages

✓ Programs larger than physical memory can run.

✓ Increases degree of multiprogramming.

✓ Efficient memory utilization.

✓ Process isolation — each process has its own virtual address space.

✓ Enables demand paging for better performance.

Disadvantages

✗ Page fault handling causes significant overhead.

✗ Thrashing degrades system performance severely.

✗ Requires additional hardware support (MMU — Memory Management Unit).

✗ Disk I/O for swapping is slow — reading from disk takes milliseconds vs nanoseconds for RAM, causing significant delay on every page fault.

★ EXAM FOCUS

- Virtual Memory allows running programs LARGER than physical RAM.
- Demand Paging: pages loaded only when accessed.
- Page Fault: accessing a page not currently in physical memory.
- Belady's Anomaly: In FIFO, adding more frames can INCREASE page faults.
- Thrashing = excessive paging, very low CPU utilization.

- Working Set Model is used to prevent thrashing.

2.4 Comparison: Segmentation vs Paging vs Virtual Memory

Segmentation	Paging	Virtual Memory
Division: Variable-size segments	**Division:** Fixed-size pages/frames	**Division:** Pages extended to disk
Fragmentation: External	**Fragmentation:** Internal	**Fragmentation:** Internal
View: Programmer's logical view	**View:** Hardware view	**View:** Extended hardware view
Address: <Seg No, Offset>	**Address:** <Page No, Offset>	**Address:** <Page No, Offset> + disk
Translation: Segment Table	**Translation:** Page Table + TLB	**Translation:** Page Table + swap space
Sharing: Easy (segment level)	**Sharing:** Possible (page level)	**Sharing:** Possible
Complexity: Moderate	**Complexity:** Low–Moderate	**Complexity:** High

3 I/O Management

Concept of I/O Port

Introduction to I/O Management

I/O Management is the OS subsystem responsible for controlling Input/Output operations between the CPU, memory, and peripheral devices. The OS provides a uniform interface for all I/O devices through device drivers and I/O ports.

3.1 Concept of I/O Port

Definition: I/O Port

An I/O port is a hardware interface through which the CPU communicates with peripheral devices. Each device is assigned a unique port address (I/O address) that the CPU uses to send commands or data to the device or read data from it.

Working Mechanism

Each I/O device has associated registers: a **Data Register** (holds data being transferred), a **Status Register** (indicates device state), and a **Control Register** (receives commands). The CPU accesses these registers using I/O port addresses via IN and OUT instructions.

Types of I/O Ports

Type	Description	Example
Serial Port	Transmits data one bit at a time; slower but long distance.	COM1, RS-232
Parallel Port	Transmits multiple bits simultaneously; faster, shorter range.	LPT1 (Printer)
USB Port	Universal (one port for all devices — keyboard, printer, phone, pen drive); supports hot-plugging (connect/disconnect while PC is ON); high speed.	USB 2.0, USB 3.0
PS/2 Port	Dedicated for keyboard/mouse; 6-pin DIN connector.	Keyboard, Mouse
VGA/HDMI	Video output ports for display devices.	Monitor connection
SATA Port	Serial ATA for storage devices.	Hard drives, SSD

Data Transfer Process

Three methods of I/O data transfer:

1. PROGRAMMED I/O (Polling/Busy-Wait):

CPU continuously checks device status register (polls) until transfer completes.

Simple but CPU is wasted during polling.

2. INTERRUPT-DRIVEN I/O:

Device sends interrupt to CPU when ready. CPU can do other work in between.

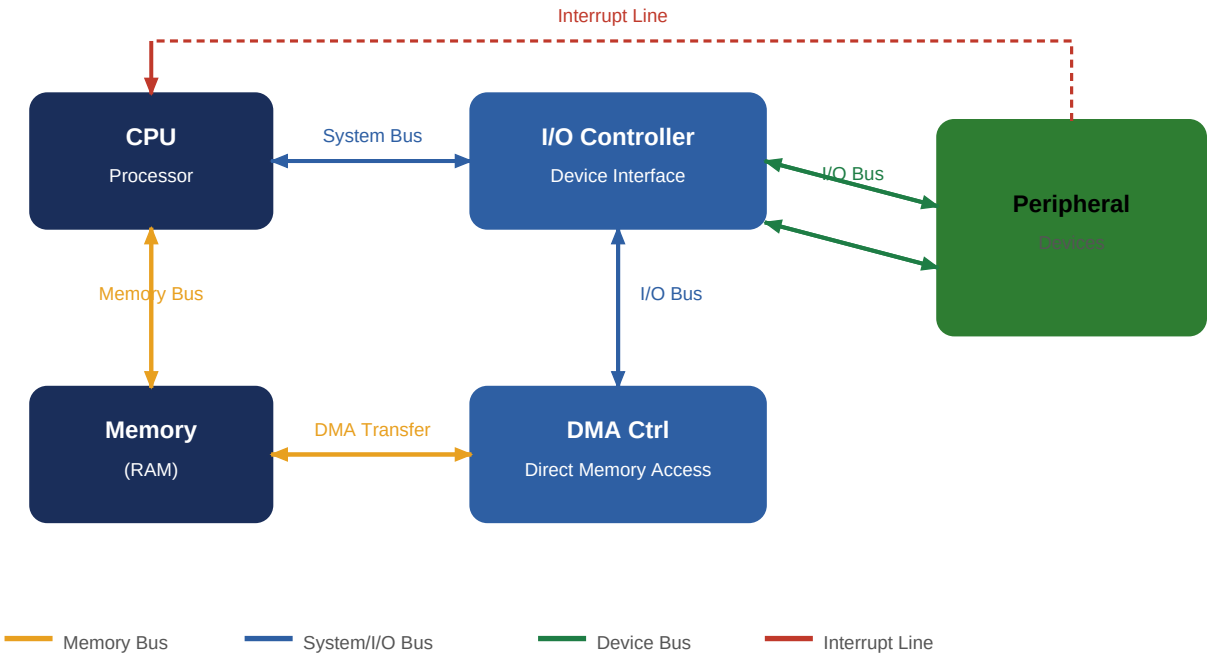
More efficient; CPU not wasted.

3. DMA (Direct Memory Access):

DMA controller handles transfer directly between device and memory.

CPU only involved at start and end of transfer. Most efficient.

Block Diagram of I/O System



Advantages & Limitations

Advantages

✓ Standard interface simplifies device programming.

✓ Device independence — same interface for all devices.

✓ Protection — user programs access devices through OS only.

✓ Buffering and caching improve performance.

Disadvantages

✗ I/O operations are slower than CPU speed.

✗ Interrupt handling adds overhead.

✗ Bus contention when multiple devices transfer simultaneously.

✗ Complex device driver development.

★ EXAM FOCUS

- I/O Ports have 3 registers: Data, Status, Control.

- Polling = CPU keeps checking; wastes CPU cycles.
- Interrupt-driven I/O = more efficient than polling.
- DMA = most efficient; CPU freed for other tasks during transfer.
- IN instruction reads from port; OUT instruction writes to port.

4

File Management

FAT | File Handling Functions

Introduction to File Management

File management is the OS function that manages files stored on secondary storage. It provides mechanisms for creating, reading, writing, deleting, and organizing files in a directory structure. The OS abstracts physical storage details from the user.

4.1 FAT – File Allocation Table

Definition: FAT (File Allocation Table)

FAT is a file system that uses a table stored near the beginning of the disk volume to track which disk clusters (groups of sectors) belong to each file. Each entry in the FAT points to the next cluster of the file, forming a linked list.

Basics: Sector and Cluster

Term	Description	Size
Sector	The smallest physical unit of storage on a disk. Data is read/written one sector at a time by the disk hardware.	Usually 512 bytes
Cluster	A group of sectors combined together. The smallest unit the OS allocates to a file. Even a 1-byte file occupies one full cluster.	4 KB (typical) = 8 × 512 byte sectors

Analogy: Think of a disk as a parking lot.

Sector = one small brick on the ground.

Cluster = one full parking space (made up of several bricks).

Even if your car (file) is small, it occupies one full parking space (cluster).

Key point: If file size < cluster size → remaining space is wasted → Internal Fragmentation.

Example: File = 1 KB, Cluster = 4 KB → 3 KB wasted per file.

Structure of FAT

The disk is divided into equal-sized **clusters**. The FAT is an array where each index corresponds to a cluster number. The value at each index indicates the next cluster of the file or a special value (EOF, Free, Bad).

FAT Entry Value	Meaning
0x000	Free cluster — available for use.
0xFFF8-0xFFFF	End of File (EOF) — last cluster of a file.

FAT Entry Value	Meaning
0xFFF7	Bad cluster — physically damaged, do not use.
Any other value	Next cluster number in the chain.

Working Process

When a file is stored on disk, the OS records its starting cluster number in the **Directory Entry**. The FAT table is then used to find all the clusters belonging to the file by following a chain from one cluster to the next until EOF is reached.

Example: A file uses clusters 5 → 8 → 12 → EOF

Directory Entry: File "report.txt" starts at Cluster 5

FAT Table:

Cluster 5 → 8 (next cluster is 8)

Cluster 8 → 12 (next cluster is 12)

Cluster 12 → EOF (end of file)

To read the file: follow the chain 5 → 8 → 12 until EOF.

✔ **KEY POINTS**

- The FAT acts like a map — it tells the OS where each piece of a file is stored on disk.
- Clusters of a file do NOT need to be stored next to each other — they can be scattered anywhere on the disk.
- This is called Non-Contiguous Allocation — the FAT chain links them together.
- When a file is deleted, its FAT entries are marked as Free (0x000) — the data is not immediately erased.
- FAT is stored twice on disk for redundancy — if one copy is corrupted, the OS uses the backup.

FAT Versions

Version	Cluster Size (bits)	Max Volume Size	Usage
FAT12	12-bit entries	~32 MB	Floppy disks, small USB drives
FAT16	16-bit entries	~2 GB	Older hard drives, compatibility
FAT32	32-bit entries (28 used)	~2 TB	USB drives, SD cards, large volumes
exFAT	64-bit entries	~128 PB	Modern flash drives, SDXC cards

Advantages & Disadvantages

Advantages

✓ Simple and well-understood structure.

✓ Compatible with virtually all operating systems.

✓ Easy to implement.

✓ Reliable — FAT is typically stored twice for redundancy.

✓ Good for removable media (USB, SD cards).

Disadvantages

✗ No built-in file permissions or security.

✗ No journaling — vulnerable to corruption on power loss.

✗ Slow for large directories (linear search).

✗ FAT32 max file size is 4 GB (limitation for large video files).

✗ Fragmentation reduces performance over time.

★ EXAM FOCUS

- FAT is stored near the BEGINNING of the disk volume.
- FAT is typically stored TWICE for redundancy.
- FAT32 maximum file size = 4 GB.
- EOF value in FAT12 = 0xFF8-0xFFF; FAT16 = 0xFFF8-0xFFFF; FAT32 = 0x0FFFFFF8-0x0FFFFFFF
- FAT uses LINKED ALLOCATION — clusters form a chain.
- Bad cluster value = 0xFF7 (FAT12), 0xFFF7 (FAT16), 0x0FFFFFF7 (FAT32)

4.2 File Handling Functions

Definition: File Handling Functions

File handling functions are OS-provided system calls and library functions that allow programs to perform operations on files. These operations include creating, opening, reading, writing, seeking, and closing files.

Common File Operations

Operation	Description	Example Call	Result
CREATE	Creates a new empty file in the file system. Allocates directory entry.	create("report.txt")	Returns file handle/descriptor or error code.
OPEN	Opens an existing file and returns a file descriptor for subsequent operations.	open("report.txt", READ)	Returns file descriptor (fd); file pointer at start.
READ	Reads specified number of bytes from current file pointer position.	read(fd, buffer, 100)	Data placed in buffer; file pointer advances.
WRITE	Writes data to file at current file pointer position.	write(fd, buffer, 100)	Data written to file; pointer advances.
APPEND	Moves file pointer to end of file and writes data.	open("file.txt", APPEND)	Ensures data is always added at the end.
SEEK	Moves file pointer to a specific byte position within the file.	seek(fd, 50, SEEK_SET)	File pointer moves to byte 50.

Operation	Description	Example Call	Result
CLOSE	Releases the file descriptor; flushes buffers to disk.	close(fd)	File descriptor released; buffers flushed.
DELETE	Removes the file from the directory and marks clusters as free.	delete("report.txt")	Directory entry removed; space reclaimed.
RENAME	Changes the name of a file in the directory entry.	rename("old.txt","new.txt")	Directory entry updated with new name.

File Open Modes

Mode	Description	File Exists	File Not Exists
r (Read)	Opens file for reading only.	Opens normally.	Error.
w (Write)	Opens for writing; truncates file.	Clears content.	Creates new.
a (Append)	Opens for writing at end.	Appends data.	Creates new.
r+ (Read/Write)	Opens for both reading and writing.	Opens normally.	Error.
w+ (Write/Read)	Read+Write; truncates file.	Clears content.	Creates new.
a+ (Append/Read)	Read+Append.	Reads anywhere; writes at end.	Creates new.

★ EXAM FOCUS

- "w" mode TRUNCATES the file (erases existing content).
- "a" mode ALWAYS writes at the end regardless of seek position.
- close() is essential — un-closed files may lose buffered data.
- File descriptor is an integer index into the OS file table.
- STDIN=0, STDOUT=1, STDERR=2 are standard file descriptors in UNIX.
- seek(fd, 0, SEEK_SET) → beginning; SEEK_CUR → current; SEEK_END → end.

5

Interrupts

Software Interrupts | Hardware Interrupts

Introduction to Interrupts

An **interrupt** is a signal to the CPU that temporarily halts the current program execution, saves its state, and transfers control to a special routine called an **Interrupt Service Routine (ISR)** or **Interrupt Handler** (a special piece of code that runs automatically to handle a specific interrupt — every interrupt has its own ISR). After the ISR completes, the CPU resumes the interrupted program.

5.1 Hardware Interrupts

Definition: Hardware Interrupt

A hardware interrupt is generated by an external hardware device (keyboard, mouse, printer, timer, network card) to signal the CPU that it requires attention. These interrupts are asynchronous — they can occur at any time during program execution.

Working Process

Step 1: Hardware device completes an operation or needs service → asserts interrupt line (IRQ).

Step 2: CPU finishes current instruction.

Step 3: CPU saves current state (PC, registers, flags) onto the stack.

Step 4: CPU reads the interrupt vector table (IVT) to find ISR address.

Step 5: CPU jumps to ISR and executes it.

Step 6: ISR services the device (e.g., reads keyboard data, acknowledges printer).

Step 7: CPU restores saved state from stack.

Step 8: CPU resumes execution of interrupted program.

IRQ Lines & Priority

IRQ (Interrupt Request) is a hardware signal wire that a device uses to notify the CPU that it needs attention (e.g., key pressed, disk read complete).

IRQ Priority: When multiple devices interrupt at the same time, the CPU handles them in order — lower IRQ number = higher priority (IRQ 0 handled first).

Analogy: Think of IRQ lines as phone lines in an office. If two departments call at the same time, the boss (CPU) picks up the most important line first (lowest IRQ number).

IRQ	Device	Priority
IRQ 0	System Timer	Highest
IRQ 1	Keyboard	High
IRQ 2	PIC Cascade	High
IRQ 3/4	COM Ports (Serial)	Medium
IRQ 6	Floppy Disk	Medium
IRQ 7	Parallel Port (Printer)	Medium
IRQ 14/15	IDE Hard Disk Controller	Lower

Types of Hardware Interrupts

Type	Description
Maskable Interrupt (INTR)	Can be disabled by CPU using CLI instruction. Non-critical.
Non-Maskable Interrupt (NMI)	Cannot be disabled. Used for critical errors (memory parity).
Reset Interrupt	System reset; highest priority hardware interrupt.

5.2 Software Interrupts

Definition: Software Interrupt

A software interrupt is generated by a program instruction (INT n in x86 assembly). It is synchronous — occurs at a defined point in program execution. Software interrupts are used to request OS services (system calls) or to handle software exceptions.

Working Process

Step 1: Program executes INT instruction (e.g., INT 21h for DOS services).

Step 2: CPU saves current state (PC, registers) on stack.

Step 3: CPU looks up interrupt number in Interrupt Vector Table (IVT).

Step 4: Control transferred to corresponding ISR (Interrupt Service Routine — code that handles the interrupt).

Step 5: ISR performs requested OS service.

Step 6: CPU restores state from stack.

Step 7: Program resumes after INT instruction.

Common Software Interrupts (x86)

Interrupt	Purpose
INT 0	Divide by Zero Exception

Interrupt	Purpose
INT 1	Single-Step Debug
INT 3	Breakpoint (Debugger)
INT 4	Overflow
INT 21h	DOS System Call (File, Console I/O)
INT 10h	BIOS Video Services
INT 13h	BIOS Disk Services
INT 80h	Linux System Call

Interrupt Vector Table (IVT)

The IVT is a table in memory (at address 0x0000 in x86 real mode) that maps interrupt numbers to their ISR addresses. Each entry is 4 bytes (2-byte segment + 2-byte offset). The IVT has 256 entries (INT 0 to INT 255).

Comparison: Hardware vs Software Interrupts

Hardware Interrupt	Software Interrupt
Source: External hardware device	**Source:** Program instruction (INT n)
Timing: Asynchronous — can occur anytime	**Timing:** Synchronous — occurs at INT instruction
Trigger: Hardware IRQ signal	**Trigger:** Software INT instruction
Maskable: Yes (INTR); No (NMI)	**Maskable:** Generally no
Priority: Based on IRQ line number	**Priority:** Based on interrupt number
Purpose: Device service requests	**Purpose:** System calls, exceptions
Examples: Keyboard, timer, disk	**Examples:** INT 21h, INT 80h, divide-by-zero

★ EXAM FOCUS

- Hardware interrupts are ASYNCHRONOUS; Software interrupts are SYNCHRONOUS.
- NMI (Non-Maskable Interrupt) cannot be disabled.
- ISR (Interrupt Service Routine) = Interrupt Handler (a special code that runs automatically to handle a specific interrupt).
- CPU saves state before jumping to ISR; restores on return.
- IVT has 256 entries (0-255); located at physical address 0 in real mode.
- INT instruction in x86 generates a software interrupt.
- Interrupt latency = time from interrupt signal to start of ISR.

6

Addressing Concepts

I/O Mapped | Memory Mapped Addressing

Introduction to Addressing

When a CPU communicates with I/O devices, it needs a mechanism to distinguish between accessing a memory location and accessing an I/O device register. Two fundamental approaches exist: **I/O Mapped (Isolated I/O)** and **Memory Mapped I/O**.

6.1 I/O Based Addresses (Isolated I/O / Port-Mapped I/O)

Definition: I/O Mapped Addressing

In I/O mapped (isolated) addressing, I/O devices and memory have separate address spaces. The CPU uses special instructions (IN/OUT in x86) to communicate with I/O devices. The I/O address space is completely separate from the memory address space.

Working Principle

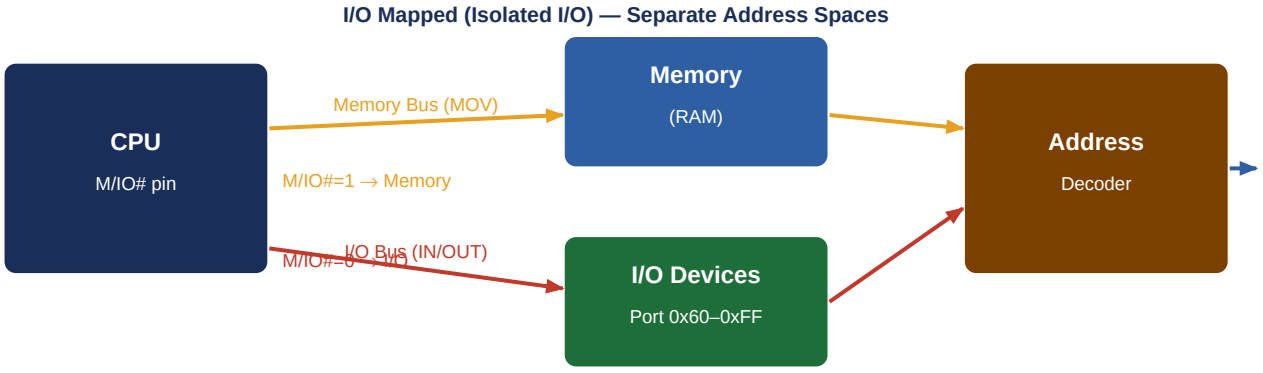
- CPU has two separate address buses: Memory Address Bus and I/O Address Bus.
- A control signal (M/IO#) indicates whether the CPU is accessing memory or I/O.
- M/IO# = 1 → Memory access (using MOV instructions)
- M/IO# = 0 → I/O access (using IN/OUT instructions)

Example (x86):

IN AL, 0x60 ; Read from I/O port 0x60 (keyboard data port)

OUT 0x61, AL ; Write to I/O port 0x61 (speaker control)

Architecture



IN/OUT instructions use a 16-bit I/O port address (0x0000 to 0xFFFF = 65,536 ports)

MOV instructions use the full memory address space (e.g., 32-bit or 64-bit)

M/IO# = 1 → Memory access | M/IO# = 0 → I/O device access

6.2 Memory Based Addresses (Memory Mapped I/O)

Definition: Memory Mapped Addressing

In memory mapped I/O, I/O device registers are mapped into the same address space as memory. The CPU uses the same instructions (MOV, LOAD, STORE) to access both memory and I/O devices. There are NO special IN/OUT instructions.

Working Principle

• A portion of the memory address space is reserved for I/O devices.

• Accessing a reserved address actually accesses an I/O device register, not RAM.

• The same data bus, address bus, and control signals are used for both memory and I/O.

Example:

Address 0xB8000 → Video RAM (text mode display memory)

Address 0xA0000 → Graphics Mode Video Memory

Writing to 0xB8000 updates what appears on the screen.

MOV [0xB8000], AX ; Write character to video memory (displays on screen)

Comprehensive Comparison

Feature	I/O Mapped (Isolated I/O)	Memory Mapped I/O
Address Space	Separate I/O address space	Shared with memory address space
Instructions	Special IN/OUT instructions	Regular MOV/LOAD/STORE
Address Range	16-bit (65,536 ports max)	Full memory address range
Memory Space	Not reduced	Reduces available memory addresses
Complexity	Hardware is more complex (M/IO# signal)	Simpler hardware design
Speed	Slightly slower (special instructions)	Faster (unified bus)
Portability	Architecture-specific (x86)	Architecture-independent
Protection	Easier to protect I/O space	Harder (same address space)
Used In	x86 PCs (IBM PC architecture)	ARM, MIPS, most embedded systems

Feature	I/O Mapped (Isolated I/O)	Memory Mapped I/O
Example Devices	Keyboard (0x60), COM1 (0x3F8)	Display memory (0xB8000)

Advantages & Disadvantages

I/O Mapped Addressing:

Advantages

✓ Separate address spaces — no memory "wasted" on I/O.

✓ Easier memory protection.

✓ Debugging easier (can see I/O access clearly).

Disadvantages

✗ Requires special CPU instructions (IN/OUT).

✗ Limited I/O address space (65,536 ports).

✗ Not portable across architectures.

Memory Mapped Addressing:

Advantages

✓ All CPU instructions can access I/O.

✓ Larger address space for devices.

✓ Architecture-independent — more portable.

Disadvantages

✗ Reduces available memory address space.

✗ Accidental writes to I/O addresses can cause problems.

✗ Cache management complications.

★ EXAM FOCUS

- I/O Mapped uses IN/OUT instructions; Memory Mapped uses MOV/LOAD/STORE.
- I/O Mapped: separate address spaces. Memory Mapped: shared address space.
- x86 uses I/O Mapped for keyboard/serial; Memory Mapped for video RAM.
- ARM processors predominantly use Memory Mapped I/O.
- Memory Mapped reduces available RAM address space.
- M/IO# control signal distinguishes memory from I/O access in isolated I/O.

7

DMA – Direct Memory Access

DMA Channels | Types | Controller

Introduction & Need for DMA

In traditional I/O operations, the CPU must supervise every byte transferred between a device and memory (programmed I/O or interrupt-driven I/O), wasting valuable CPU cycles. **DMA (Direct Memory Access)** solves this by allowing I/O devices to transfer data directly to/from memory without continuous CPU involvement.

Definition: DMA (Direct Memory Access)

DMA is a feature that allows hardware peripherals (disk, NIC, sound card) to directly read from or write to main memory without CPU intervention for each byte transferred. A dedicated **DMA Controller (DMAC)** manages the transfer.

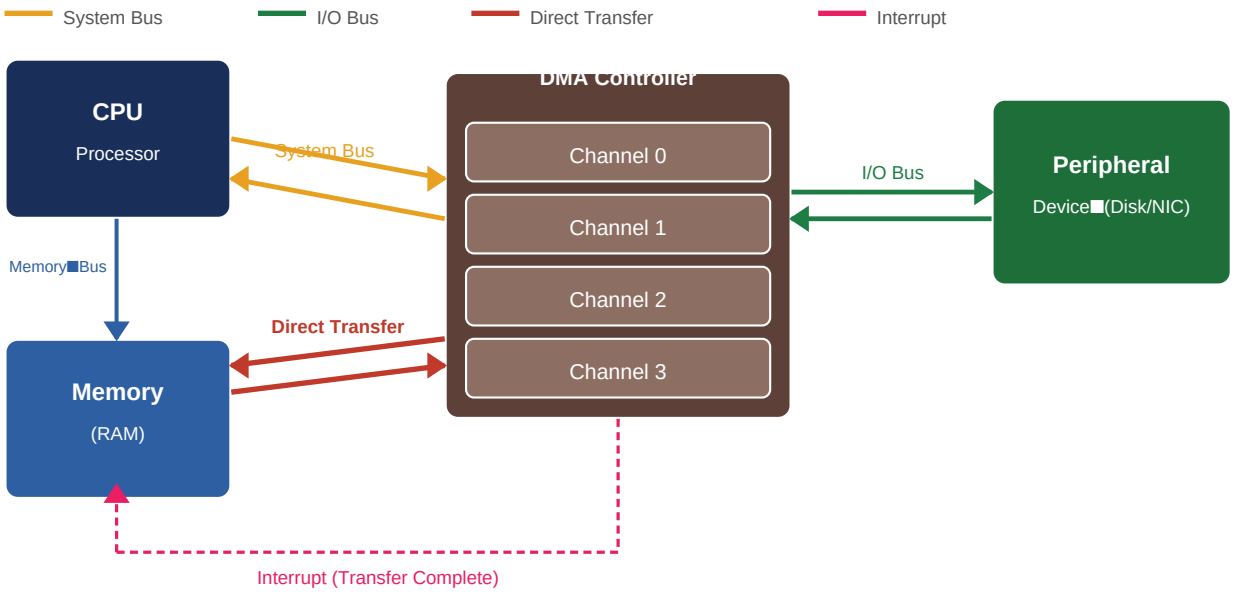
7.1 DMA Controller (DMAC)

The DMA Controller is a hardware component that manages DMA transfers. It contains multiple DMA channels, each dedicated to a specific device or transfer. The DMAC communicates with both the CPU and devices via the system bus.

DMAC Internal Registers (per channel)

Register	Purpose
Memory Address Register	Holds the starting memory address for the transfer.
Count Register	Number of bytes/words to transfer.
Control Register	Direction of transfer (read/write), transfer mode.
Status Register	Current state of the DMA channel (busy, done, error).

DMA Block Diagram



Key: DMA takes control of address, data, and control buses during transfer.

CPU is only involved at START (programming DMAC) and END (receiving interrupt).

7.2 DMA Transfer Process

Step 1: CPU programs the DMAC:

• Source/destination memory address

• Transfer count (number of bytes)

• Direction (device→memory or memory→device)

• DMA channel to use

Step 2: CPU issues DMA request and continues other tasks.

Step 3: DMAC requests bus access from CPU (via HOLD/BUSRQ signal).

Step 4: CPU releases bus (acknowledges via HLDA/BUSAK signal) → CPU stalls briefly.

Step 5: DMAC takes control of address bus, data bus, and control bus.

Step 6: DMAC transfers data directly between device and memory.

DMAC auto-increments address and decrements count after each transfer.

Step 7: When count reaches 0, transfer is complete.

Step 8: DMAC releases bus control back to CPU.

Step 9: DMAC sends interrupt to CPU to signal completion.

Step 10: CPU acknowledges interrupt and processes the transferred data.

7.3 DMA Channels

The 8237 DMA controller (used in original IBM PC) has **4 channels (0-3)**. The PC/AT system combined two 8237 chips for a total of **8 channels (0-7)**. Each channel is assigned to a specific device.

DMA Channel	Traditional Assignment	Data Width
Channel 0	DRAM Refresh (in old systems)	8-bit
Channel 1	Sound Card	8-bit
Channel 2	Floppy Disk Controller	8-bit
Channel 3	Parallel Port (ECP mode)	8-bit
Channel 4	Cascade (links two 8237 chips)	16-bit (address × 2)
Channel 5	Sound Card (16-bit)	16-bit
Channel 6	Available/Network Card	16-bit
Channel 7	Hard Disk (older systems)	16-bit

7.4 Types of DMA Transfer

Type	Description	Advantage	Disadvantage
Burst Mode	DMAC takes bus control and transfers entire block at once.	Fast — transfers complete quickly.	CPU is locked out for the entire duration.
Cycle Stealing Mode	DMAC steals one bus cycle from CPU, transfers one word, releases bus.	CPU continues with minimal interruption.	Slower — transfer takes more time.
Transparent Mode (Block Transfer)	DMAC transfers data only when CPU is not using bus.	No CPU interference at all.	Transfer time unpredictable; complex logic required.
Single Transfer Mode	One data item transferred per DMA request.	Fine-grained control.	Slower for large transfers.

Advantages & Disadvantages

Advantages

✓ CPU is free during data transfer — significantly improves system performance.

✓ High data transfer rates achievable.

✓ Essential for devices generating large amounts of data (disk, network).

✓ Reduces interrupt overhead.

✓ Multiple channels allow simultaneous device operations.

Disadvantages

✗ Additional hardware cost (DMA Controller chip).

✗ Bus contention between CPU and DMAC.

✗ Cache coherence issues — CPU cache may hold stale data after DMA.

✗ Complex initialization and configuration.

✗ In burst mode, CPU is stalled for the entire transfer.

Applications of DMA

- Hard Disk Controllers (ATA/SATA) — high-speed disk read/write.
- Network Interface Cards (NIC) — packet reception/transmission.
- Sound Cards — continuous audio stream transfer.
- USB Controllers — data transfer to/from USB devices.
- GPU Data Transfer — large framebuffer transfers.
- Video Capture Cards — continuous video frame acquisition.

★ EXAM FOCUS

- DMA allows data transfer between device and memory WITHOUT CPU intervention.
- HOLD/HLDA (or BUSRQ/BUSAK) signals coordinate bus access between CPU and DMAC.
- Three modes: Burst (fastest, CPU locked), Cycle Steal (balanced), Transparent (no interference).
- DMA can cause CACHE COHERENCE problems — CPU cache may have stale data.
- PC/AT has 8 DMA channels (Channels 0-7); Channel 4 is cascade.
- DMA transfer ends with INTERRUPT to CPU signaling completion.
- DMAC needs: starting address, count, direction, and channel to operate.

8 Glossary of Terms

Key Definitions

Aging	Technique of gradually increasing the priority of waiting processes to prevent starvation.
Belady's Anomaly	In FIFO page replacement, increasing frames may increase page faults.
Burst Time (BT)	The total CPU time required by a process to complete execution.
Cache Coherence	Ensuring CPU cache reflects current memory content (relevant after DMA).
Cluster	A group of disk sectors; the smallest allocation unit in FAT file system.
Completion Time (CT)	The time at which a process finishes execution.
Convoy Effect	Short processes stuck behind a long process in FCFS, causing high wait times.
Context Switch	Saving CPU state of one process and loading state of another.
Demand Paging	Loading pages into memory only when they are accessed (on demand).
DMA (Direct Memory Access)	Data transfer between device and memory without CPU intervention.
DMAC (DMA Controller)	Hardware chip that manages DMA transfers.
EOF (End of File)	Special FAT entry value indicating the last cluster of a file.
External Fragmentation	Wasted memory space between allocated segments; cannot be used.
FAT (File Allocation Table)	Table tracking which disk clusters belong to which files.
File Descriptor	Integer index returned by open(); used to reference an open file.
Frame	Fixed-size block of physical memory in paging.
IVT (Interrupt Vector Table)	Table mapping interrupt numbers to ISR addresses.
Internal Fragmentation	Wasted memory inside an allocated block due to fixed-size allocation.
Interrupt	Signal causing CPU to stop current task and run an ISR.
IRQ (Interrupt Request)	Hardware signal line used by devices to interrupt the CPU.
ISR (Interrupt Service Routine)	Code that handles an interrupt; also called interrupt handler.
Logical Address	Address generated by CPU; translated to physical address by MMU.
MMU (Memory Management Unit)	Hardware that translates logical addresses to physical addresses.
NMI (Non-Maskable Interrupt)	Hardware interrupt that cannot be disabled.
Page	Fixed-size block of logical memory in paging.
Page Fault	Occurs when a requested page is not in physical memory.
Page Table	OS data structure mapping page numbers to frame numbers.
Physical Address	Actual address in RAM; result of address translation.
Preemptive Scheduling	CPU can be taken from a running process before it finishes.
Priority Inversion	Low-priority process holds resource needed by high-priority process.

Segmentation	Dividing a process into variable-size logical segments.
Segment Table	Maps segment numbers to base address and limit.
Starvation	A process waits indefinitely because higher-priority processes always preempt it.
Swap Space	Disk area used to store pages swapped out of physical memory.
Thrashing	System spends more time paging than executing; caused by insufficient frames.
Time Quantum (TQ)	Fixed time slice allocated to each process in Round Robin scheduling.
TLB (Translation Lookaside Buffer)	Fast cache for recent page table entries.
Turnaround Time (TAT)	Total time from process arrival to completion. TAT = CT - AT.
Virtual Memory	Technique allowing programs to use more memory than physically available.
Waiting Time (WT)	Time a process spends waiting in the ready queue. WT = TAT - BT.

9

Revision Summary & Conclusion

Quick Review of All Topics

1. Process Management

- FCFS: Non-preemptive; FIFO order; causes Convoy Effect.
- Round Robin: Preemptive; fixed Time Quantum; best response time; no starvation.
- Priority: Preemptive or non-preemptive; starvation solved by Aging.
- Key formulas: TAT = CT - AT | WT = TAT - BT | Avg WT = sum/n

2. Memory Management

- Segmentation: Variable-size segments; external fragmentation; addresses.
- Paging: Fixed-size pages/frames; internal fragmentation; addresses.
- Virtual Memory: Demand paging; page fault handled by OS; thrashing is a problem.
- TLB speeds up paging. Aging/Working Set prevents thrashing.

3. I/O Management

- I/O Port: Interface for CPU-device communication; Data, Status, Control registers.
- Three I/O methods: Polling (wastes CPU), Interrupt-driven (efficient), DMA (best).
- IN reads from port; OUT writes to port.

4. File Management

- FAT: Linked cluster allocation table; stored twice; FAT32 max file = 4 GB.
- File operations: Create, Open, Read, Write, Append, Seek, Close, Delete.
- "w" truncates; "a" appends; "r" reads only.

5. Interrupts

- Hardware: Asynchronous; from devices; uses IRQ lines; maskable (INTR) and non-maskable (NMI).
- Software: Synchronous; from INT instruction; used for system calls and exceptions.
- IVT maps interrupt numbers to ISR addresses. CPU saves/restores state around ISR.

6. Addressing Concepts

- I/O Mapped: Separate address spaces; IN/OUT instructions; 65,536 ports (x86).
- Memory Mapped: Shared address space; regular MOV instructions; used in ARM/embedded.
- Key difference: I/O mapped uses M/IO# signal; Memory mapped does not.

7. DMA

- DMA: Device transfers data directly to/from memory without CPU per-byte involvement.
- DMAC has per-channel registers: Address, Count, Control, Status.
- Modes: Burst (CPU locked), Cycle Steal (CPU shares bus), Transparent (no interference).
- PC/AT has 8 DMA channels. Channel 4 is cascade. Transfer ends with CPU interrupt.

Final Tips for Exam Success

1. ALWAYS write formulas first: TAT = CT - AT, WT = TAT - BT.
2. Draw Gantt charts clearly with time markers for scheduling questions.
3. For paging address translation: clearly separate Page Number and Offset.
4. For segmentation: always CHECK if Offset < Limit before computing physical address.
5. Remember: FCFS = No starvation, No preemption | RR = Preemptive | Priority = Starvation possible.
6. DMA ≠ interrupts; DMA handles data transfer; interrupts signal completion.
7. FAT stored TWICE for redundancy. Max FAT32 file size = 4 GB.
8. Hardware interrupt = ASYNC (from device). Software interrupt = SYNC (from INT instruction).
9. Memory Mapped = regular instructions. I/O Mapped = IN/OUT instructions.
10. Virtual Memory: Page fault ≠ error; it is handled by OS. Thrashing = too many page faults.

Conclusion

This study guide has covered all the topics in the Operating System Introduction syllabus in a structured, exam-focused manner. Each section includes definitions, working principles, examples, comparison tables, and exam notes designed to reinforce understanding. Regular revision of the key formulas, Gantt chart construction, address translation processes, and comparison tables will ensure thorough preparation for university and competitive examinations. Good luck!